Chapter 1: Linear Regression & Logistic Models

Zero-to-Research Machine Learning Handbook — A First Course for Starting Students

🔗 *Further reading: Course text: Chapter 1 (this handbook)*

Chapter 1 Roadmap

- Part 0 — Foundations: the algebra, logs, probability, and Python you need first
- Part I — Mathematical Foundations: what regression is and how it is derived
- Part II — From Scratch: building regression in pure Python, no libraries
- Part III — Libraries: NumPy, pandas, statsmodels, scikit-learn
- Part IV — Exercises: conceptual → derivation → coding → library tiers
- Part V — Research-Level Practice: what comes after you're comfortable

💡 ***Key insight: Each part builds on the last — do not skip Part 0 even if it looks basic.***

🔗 *Further reading: How to use this guide (full chapter)*

Math Refresher: Variables, Slope, and Intercept

- A variable is a symbol that can change: x (rainfall), y (crop yield)
- A line is written $y = a + b \cdot x$
- a is the intercept: the value of y when x = 0
- b is the slope: how much y changes per one-unit increase in x
- Rearranging: given y, a, b → solve for $x = (y - a) / b$

💡 ***Key insight: Every regression coefficient you will ever fit is just a slope, generalized to many predictors at once.***

🔗 *Further reading: Khan Academy: slope & linear equations*

Math Refresher: Logs, Exponentials, and Odds

- Exponential growth multiplies by a fixed factor each step: 2, 4, 8, 16, …
- A logarithm undoes an exponential: log(exp(x)) = x
- Log rules: log(ab) = log(a)+log(b); log(a/b) = log(a)−log(b); log(a^c) = c·log(a)
- Odds = p / (1−p); log-odds ("logit") can be any real number
- Logistic regression models the logit as a straight line — same tool, new scale

💡 ***Key insight: Logs turn products into sums — this is exactly why log-likelihood, not likelihood, is used to fit every model in this handbook.***

🔗 *Further reading: Wikipedia: Logarithm*

Statistics Refresher: Population, Sample, Mean, Variance

- Population: the entire group you want to learn about
- Sample: the smaller group you actually observed
- Mean: add the values, divide by the count
- Variance: how spread out values are around the mean
- Covariance: whether two variables move together
- Correlation: covariance rescaled to always fall between −1 and 1

💡 ***Key insight: The simple regression slope is nothing more than covariance(x,y) divided by variance(x).***

🔗 *Further reading: Seeing Theory: visual intro to statistics*

Python Refresher: What You Need to Read the Code

- Lists store multiple values: x = [1, 2, 3, 4, 5]
- Loops repeat work: for value in x: …
- Functions name reusable logic: def mean(values): return sum(values)/len(values)
- zip() pairs values from two lists together
- List comprehensions build a new list in one line

💡 ***Key insight: You do not need advanced Python for Chapter 1 — lists, loops, and functions are enough to read every line of code.***

🔗 *Further reading: Official Python tutorial*

What Is Regression, Really?

- Description: summarize how y relates to x in the data you have
- Prediction: guess y for a new observation's x
- Explanation: estimate how much x influences y, holding other things fixed
- Causal inference: what happens to y if we intervene and change x?
- These four goals require different levels of assumption — don't mix them up

💡 ***Key insight: A model that predicts well is not automatically a model that explains or proves causation.***

🔗 *Further reading: Wikipedia: Correlation does not imply causation*

A Tiny Regression, By Hand

- $x = [1, 2, 3, 4, 5]$, $y = [2, 4, 5, 4, 5]$
- $\bar{x} = 3$, $\bar{y} = 4$
- slope $= \Sigma(x_i - \bar{x})(y_i - \bar{y}) \,/\, \Sigma(x_i - \bar{x})^2 = 6/10 = 0.6$
- intercept $= \bar{y} - \text{slope} \cdot \bar{x} = 4 - 0.6(3) = 2.2$
- fitted line: $\hat{y} = 2.2 + 0.6x$

💡 ***Key insight: At x=4, the prediction is 4.6; the observed value is 4, so the residual is −0.6.***

🔗 *Further reading: StatQuest: Linear Regression, Clearly Explained (YouTube channel)*

Simple Linear Regression: The Model

- Model: $y = \beta_0 + \beta_1 x + \varepsilon$
- ε (epsilon) is the unobservable disturbance in the true process
- e (the residual) is what we actually observe: $e = y - \hat{y}$
- Least squares chooses β_0, β_1 to minimize Σe_i^2
- Why squared error? It penalizes large mistakes more, and has clean calculus

💡 ***Key insight: ε and e are not the same thing — e is our best estimate of ε, not ε itself.***

🔗 *Further reading: Wikipedia: Ordinary least squares*

Deriving the Slope and Intercept

- Take the derivative of $\Sigma(y_i - b_0 - b_1 x_i)^2$ with respect to b_0 and b_1
- Set both derivatives to zero → the two "normal equations"
- Solving them gives: $b_1 = S_{xy} / S_{xx}$ and $b_0 = \bar{y} - b_1 \bar{x}$
- $S_{xy} = \Sigma(x_i - \bar{x})(y_i - \bar{y})$; $S_{xx} = \Sigma(x_i - \bar{x})^2$
- The fitted line always passes through the point $(\bar{x}, \bar{y})$

💡 ***Key insight: This is not magic — it is one application of "set the derivative to zero to find a minimum," the same rule from introductory calculus.***

🔗 *Further reading: 3Blue1Brown: essence of calculus*

Multiple Linear Regression & the Design Matrix

- Real problems have many predictors: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \varepsilon$
- Stack all predictors into a design matrix X (rows = observations, columns = predictors)
- Model in matrix form: $y = X\beta + \varepsilon$
- Least squares solution: $\hat{\beta} = (X^T X)^{-1} X^T y$ — the "normal equations"
- Each β_j is interpreted "holding all other predictors constant"

💡 ***Key insight: Matrix notation is not a new idea — it is simple regression's algebra, just packing many x's into one object.***

🔗 *Further reading: Wikipedia: Design matrix*

The Geometry of OLS

- $X\hat{\beta}$eta is the projection of y onto the space spanned by X's columns
- The residual vector e is orthogonal (perpendicular) to every column of X: $X^T e = 0$
- This is why an intercept forces the residuals to sum to exactly zero
- Geometrically: OLS finds the closest point to y that X can possibly reach

💡 ***Key insight: "Closest point" is measured in squared Euclidean distance — this is exactly why the method is called least squares.***

🔗 *Further reading: Wikipedia: Projection (linear algebra)*

The Assumptions Behind OLS

- Linearity: the true relationship is (approximately) linear in the parameters
- Independence: observations don't influence each other
- No perfect multicollinearity: predictors aren't exact linear combinations of each other
- Zero conditional mean: $E[\varepsilon \mid X] = 0$ — no systematic error given the predictors
- Homoskedasticity: constant error variance across all values of X
- Normality: needed for exact small-sample inference (less critical with large n)

💡 ***Key insight: Each assumption supports one specific guarantee — break one, and only that guarantee is at risk, not the whole model.***

🔗 *Further reading: Wikipedia: Gauss–Markov theorem*

Sampling Uncertainty: Standard Errors, CIs, and Tests

- Every coefficient estimate would differ slightly in a fresh sample — that's sampling variation
- The standard error measures how much a coefficient would typically vary across samples
- A confidence interval reports a plausible range for the true coefficient
- A p-value asks: how surprising is this data if the true effect were zero?
- 95% confidence describes the reliability of the *procedure*, not this one interval

💡 ***Key insight: A small p-value is evidence against 'no effect' — it is not proof the effect is large or important.***

🔗 *Further reading: Wikipedia: Confidence interval*

Measuring Fit: R^2 and Adjusted R^2

- $R^2 = 1 - (SS_{residual} / SS_{total})$: proportion of variance explained by the model
- $R^2 = 0$ means the model does no better than predicting the mean every time
- Adding any predictor can only increase (never decrease) raw R^2
- Adjusted R^2 penalizes extra predictors that don't earn their keep
- A low R^2 does not mean a coefficient is unimportant (see Exercise C7)

💡 ***Key insight: A randomized trial can have a huge, precisely estimated effect and still a low R^2, if individual outcomes are just noisy.***

🔗 *Further reading: Wikipedia: Coefficient of determination*

Residual Diagnostics

- Residuals vs. fitted values: look for curved patterns (non-linearity) or funnel shapes (heteroskedasticity)
- Q-Q plot: checks whether residuals follow a normal distribution
- Leverage: how unusual a point's predictor values are
- Cook's distance: how much removing one point would change the fitted model
- A model that 'runs' without errors can still badly violate its assumptions — always plot the residuals

💡 ***Key insight: Diagnostics are not optional extra credit — they are how you find out whether the assumptions from two slides ago actually hold.***

🔗 *Further reading: Wikipedia: Anscombe's quartet*

Multicollinearity

- Occurs when predictors are highly correlated with each other
- Individual coefficients become unstable and hard to interpret
- But predicted values ($X\hat{b}eta$) usually remain reliable
- Variance Inflation Factor (VIF) quantifies how much a coefficient's variance is inflated
- VIF above roughly 5–10 is commonly flagged as concerning

💡 ***Key insight: Multicollinearity damages interpretation far more than it damages prediction — know which goal you actually care about.***

🔗 *Further reading: Wikipedia: Variance inflation factor*

Regularization: Ridge, Lasso, Elastic Net

- Ridge adds a penalty $\lambda\Sigma\beta_j^2$ — shrinks coefficients toward zero, keeps them all
- Lasso adds a penalty $\lambda\Sigma|\beta_j|$ — can shrink coefficients to exactly zero (feature selection)
- Elastic Net blends both penalties
- Regularization requires standardized features — otherwise the penalty is unfair across scales
- More regularization = more bias, less variance = a deliberate trade-off, not free lunch

💡 ***Key insight: Ridge is smooth and keeps correlated predictors together; Lasso's sharp corners tend to arbitrarily pick one from a correlated group and zero out the rest.***

🔗 *Further reading: Wikipedia: Regularization (mathematics)*

Logistic Regression: From Probability to Log-Odds

- Outcome y is binary (0 or 1) — a straight line can't stay between 0 and 1
- Model the log-odds instead: $\text{logit}(p) = \log(p/(1-p)) = \beta_0 + \beta_1 x$
- Invert to get a probability: $p = 1 / (1 + \exp(-(\beta_0+\beta_1 x)))$
- $\exp(\beta_j)$ is the multiplicative change in the ODDS, not the probability, per unit of x
- Fit by maximum likelihood, not least squares — no closed-form solution, needs iteration

💡 ***Key insight: Logistic regression is still, at its core, a straight line — just measured on the log-odds scale instead of the raw probability scale.***

🔗 *Further reading: Wikipedia: Logistic regression*

Evaluating Classification Models

- Confusion matrix: true/false positives and negatives at a chosen threshold
- Precision: of predicted positives, how many were correct?
- Recall: of actual positives, how many did we find?
- Accuracy is misleading when classes are imbalanced (e.g., 2% positive rate)
- ROC-AUC and precision-recall AUC summarize performance across all thresholds

💡 ***Key insight: A model that always predicts 'no' can score 98% accuracy on a 2%-positive dataset while being completely useless.***

🔗 *Further reading: Wikipedia: Precision and recall*

Regression From Scratch, in Pure Python

- No NumPy, pandas, or scikit-learn — only the Python standard library
- Implements: mean/variance, simple & multiple OLS, gradient descent, ridge, lasso
- Also includes: Student-t distribution, confidence & prediction intervals
- Train/test split and k-fold cross-validation, all built by hand
- Goal: nothing is a black box — every library call later maps to code you've already written

💡 ***Key insight: If you can explain every line of the from-scratch module, you can never be fooled by a library's default settings.***

🔗 *Further reading: Real Python: linear regression in Python*

Regression With Python Libraries

- NumPy: the linear algebra engine underneath almost everything else
- pandas: loading, cleaning, and preparing tabular data
- statsmodels: rich statistical inference — coefficients, SEs, p-values, diagnostics
- scikit-learn: fast, consistent pipelines for prediction and cross-validation
- Rule of thumb: statsmodels for inference, scikit-learn for prediction pipelines

💡 ***Key insight: Different libraries exist because 'explain the data' and 'predict on new data' are different jobs, even when the underlying model is the same.***

🔗 *Further reading: scikit-learn: linear models user guide*

Common Mistakes and Corrections

- “The coefficient is significant, so the effect is important” → report magnitude & units too
- “Scale the full dataset, then split” → fit scaling only inside the training fold
- “Use the test set until the model looks good” → test set becomes training info if reused
- “A 0.5 threshold is always correct” → choose thresholds from costs and prevalence
- “Delete every outlier” → investigate first; unusual points can be genuine and informative

💡 ***Key insight: Almost every 'common mistake' in this chapter is a data-leakage or overclaiming mistake, not an arithmetic one.***

🔗 *Further reading: scikit-learn: data leakage guidance*

Practicing What You've Learned: The Exercise Tiers

- Conceptual (C): explain the idea in plain language, no formula allowed
- Derivation (D): reconstruct the mathematics from first principles, on paper
- Pure-Python coding (P): implement it using only the standard library
- Library (L): reproduce the result with NumPy, statsmodels, and scikit-learn
- Work one topic through all four tiers before moving to the next topic

💡 ***Key insight: If you can't answer a topic's Conceptual question, do not attempt its Derivation exercise yet — the tiers are a ladder, not a menu.***

🔗 *Further reading: Chapter 1 exercises with answer keys*

Causal Caution: Correlation Is Not Causation

- Ice cream sales strongly predict drowning deaths — both driven by hot weather
- A confounder is an unmeasured factor that drives both x and y
- Omitted-variable bias: leaving out a confounder correlated with x biases $\hat{\beta}$
- High predictive accuracy does NOT imply a safe intervention ("if we change x, y will change")
- Causal claims need a causal design: randomization, natural experiments, or explicit assumptions

💡 ***Key insight: A regression coefficient answers 'how are x and y associated in this data?' — not 'what happens if I change x?' unless you can defend that extra assumption.***

🔗 *Further reading: Wikipedia: Confounding*

Wrap-Up and Where to Go Next

- You now have the vocabulary: slope, residual, assumption, standard error, regularization
- You've seen regression built twice — once from scratch, once with libraries
- You know how to evaluate a model honestly, and how to avoid the most common mistakes
- Next: Chapter 2 extends this exact framework to counts, rates, and other outcome types (GLMs)
- Practice loop: pick a topic, work C → D → P → L, then move to the next topic

💡 ***Key insight: Everything in Chapter 2 (and beyond) is this same regression framework, generalized — master this chapter and the rest gets easier, not harder.***

🔗 *Further reading: Generalized Linear Models (Wikipedia, Chapter 2 preview)*